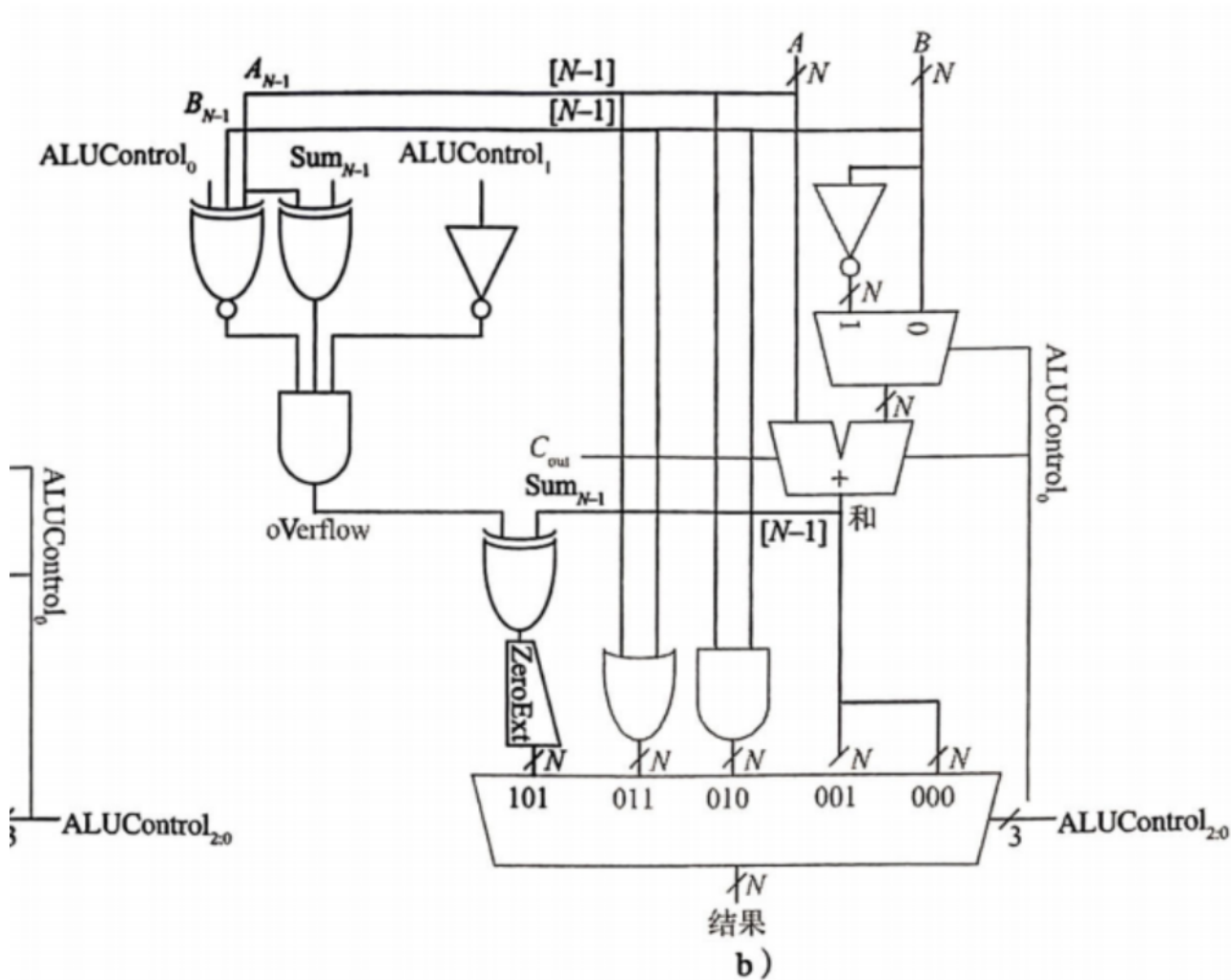


Homework Chapter5

张睿恒 2024302182001

T5.14&5.18

思路：一个考虑溢出的ALU结构如图所示：



支持 SLT 的扩展 ALU。a) 不考虑溢出。b) 考虑溢出

据图，可整理出ALU支持的操作如下表：

ALU Ctrl	op
000	ADD
001	SUB
010	AND
011	OR
101	zero extended

同时可以看出，减法的实现其实就是先将B取反再相加，所以定义了一个logic b_in来表示取反。
完整的代码如下：

```

module ALU #(parameter N=32) (
    input logic [N-1:0] a,
    input logic [N-1:0] b,
    input logic [2:0] ctrl,
    output logic [N-1:0] res,
    output logic zero,
    output logic overflow
);

logic [N-1:0] b_in;
logic [N-1:0] sum;
logic c;
logic addSub;

assign addSub = ctrl[0];
assign b_in = b^{N{addSub}};
assign {c,sum} = a+b_in+addSub;
assign overflow = (a[N-1]&b[N-1]&~sum[N-1]) | (~a[N-1]&~b[N-1]&sum[N-1]);

always_comb begin
    case(ctrl)
        3'b000: res = a&b;
        3'b001: res = a|b;
        3'b010: res = sum;
        3'b011: res = ~(a|b);
        3'b101: res = (sum==0)?0:1;
        3'b110: res = sum;
        3'b111: res = {{N-1{1'b0}},sum[N-1]};
        default: res = 0;
    endcase
end

assign zero = (res == 0);

endmodule

```

给出对应的测试代码

```

`timescale 10ps/1ps

module tb_ALU();

parameter N = 32;

logic [N-1:0] A,B,res;
logic [2:0] ctrl;
logic zero,overflow;

ALU #(N) dut(
    .a(A),
    .b(B),
    .res(res),
    .ctrl(ctrl),
    .zero(zero),
    .overflow(overflow)
);

task print_result(string op);
    $display("[%0t] %s: A=%0d, B=%0d -> Result=%0d, Zero=%b, Overflow=%b",
        $time, op, A, B, res, zero, overflow);
endtask

initial begin
    $display("==== ALU test ====");

    // AND
    A = 32'd15; B = 32'd9; ctrl = 3'b000;
    #5; print_result("AND");

    // OR
    A = 32'd15; B = 32'd9; ctrl = 3'b001;
    #5; print_result("OR");

    // ADD
    A = 32'd100; B = 32'd23; ctrl = 3'b010;
    #5; print_result("ADD");

    // SUB
    A = 32'd100; B = 32'd23; ctrl = 3'b110;
    #5; print_result("SUB");

    // SUB negative
    A = 32'd23; B = 32'd100; ctrl = 3'b110;
    #5; print_result("SUB (负数)");

    // SLT (A<B)

```

```

    A = 32'd10; B = 32'd20; ctrl = 3'b111;
    #5; print_result("SLT");

    // SLT (A>B)
    A = 32'd20; B = 32'd10; ctrl = 3'b111;
    #5; print_result("SLT");

    // NOR
    A = 32'h0F0F; B = 32'h00FF; ctrl = 3'b011;
    #5; print_result("NOR");

    // test Zero
    A = 32'd0; B = 32'd0; ctrl = 3'b101;
    #5; print_result("ZeroExtend");

    $display("==== test end ====");
    $finish;

end

endmodule

```

给出仿真时测试截图

```

# Loading work.ALU
# Load canceled
ModelSim> run -all
# ===== ALU test =====
# [50] AND: A=15, B=9 -> Result=9, Zero=0, Overflow=0
# [100] OR: A=15, B=9 -> Result=15, Zero=0, Overflow=0
# [150] ADD: A=100, B=23 -> Result=123, Zero=0, Overflow=0
# [200] SUB: A=100, B=23 -> Result=123, Zero=0, Overflow=0
# [250] SUB (8'm): A=23, B=100 -> Result=123, Zero=0, Overflow=0
# [300] SLT: A=10, B=20 -> Result=1, Zero=0, Overflow=1
# [350] SLT: A=20, B=10 -> Result=0, Zero=1, Overflow=0
# [400] NOR: A=3855, B=255 -> Result=4294963200, Zero=0, Overflow=0
# [450] ZeroExtend: A=0, B=0 -> Result=0, Zero=1, Overflow=0
# ===== test end =====
# ** Note: $finish      : /home/zrheng/Documents/DDCA2025Fall/Homework_Chap5/tb_T5.14.sv(64)
# Time: 450 ps  Iteration: 0  Instance: /tb_ALU
# 1
# Break in Module tb_ALU at /home/zrheng/Documents/DDCA2025Fall/Homework_Chap5/tb_T5.14.sv line 64

```

T5.19

思路：由于是无符号数比较大小，我们只关心C（结果是否进位）Z（结果是否为0）。容易看出C，Z与HS，LS，HI，LO的关系如下表所示：

signal	logic
HS	C
LO	$\sim C$
HI	$C \& \sim Z$
LS	$\sim C Z$ 解释一下为何大于等于可以用C来表示：由于 $A - B = A + (\tilde{B}) + 1 = A + 2^n - B = 2^n + A - B$ 对于n位加法器来说，若结果 $\geq 2^n$ ，则产生进位，同时表示 $A - B \geq 0$ ，即 $A \geq B$ 。同理后面三个也就很好理解了。下面给出代码 <pre> module unsigned_cmp(input logic C, input logic Z, output logic HS, output logic LS, output logic HI, output logic LO); assign HS = C; assign LO = $\sim C$; assign HI = $C \& \sim Z$; assign LS = $\sim C Z$; endmodule </pre>

很简单的一段代码。下面给出测试

```

`timescale 10ps/1ps

module tb_unsigned_cmp;

    logic C;
    logic Z;
    logic HS;
    logic LS;
    logic HI;
    logic LO;

    unsigned_cmp dut (
        .C(C),
        .Z(Z),
        .HS(HS),
        .LS(LS),
        .HI(HI),
        .LO(LO)
    );

    initial begin
        $display("C Z | HS LS HI LO");
        $display("-----");

        for (int i = 0; i < 4; i++) begin
            C = i[1];
            Z = i[0];
            #1;
            $display("%b %b | %b %b %b %b", C, Z, HS, LS, HI, LO);
        end

        $finish;
    end

endmodule

```

```

ModelSim> vsim work.tb_unsigned_cmp
# vsim work.tb_unsigned_cmp
# Start time: 15:16:42 On Oct 30, 2025
# Loading sv_std.std
# Loading work.tb_unsigned_cmp
# Loading work.tb_unsigned_cmp
add wave -position insertpoint sim:/tb_unsigned_cmp/dut/*
VSIModel> run -all
# C Z | HS LS HI LO
# -----
# 0 0 | 0 1 0 1
# 0 1 | 0 1 0 1
# 1 0 | 1 0 1 0
# 1 1 | 1 1 0 0
# ** Note: $finish : /home/zrheng/Documents/DDCA2025Fall/Homework_Chap5/tb_T5.19.sv(32)
# Time: 40 ps Iteration: 0 Instance: /tb_unsigned_cmp
# 1
# Break in Module tb_unsigned_cmp at /home/zrheng/Documents/DDCA2025Fall/Homework_Chap5/tb_T5.19.sv line 32

```



T5.20

思路：和5.19类似，只不过变为了有符号数。这时我们就不再关心C了。但需要注意：如果出现溢出的情况，会让N反转，所以要把N修正成为 $N^{\wedge}V$ 。具体关系如下表所示：

signal	logic
GT	$(\sim N^{\wedge}V) \& \sim Z$
LT	$N^{\wedge}V$
GE	$\sim(N^{\wedge}V)$
LE	$(N^{\wedge}V) \mid Z$

对照着这个表格写代码就很简单了。


```
module signed_cmp(  
    input logic N,  
    input logic Z,  
    input logic V,  
    output logic GT,  
    output logic LT,  
    output logic GE,  
    output logic LE  
);  
  
assign GT = (~N^V) & ~Z;  
assign LT = (N^V);  
assign GE = ~(N^V);  
assign LE = (N^V) | Z;  
  
endmodule
```

接下来给出测试：

```

`timescale 10ps/1ps

module tb_signed_cmp;
    logic N, Z, V;
    logic GE, LE, GT, LT;

    signed_cmp uut (
        .N(N), .Z(Z), .V(V),
        .GE(GE), .LE(LE), .GT(GT), .LT(LT)
    );

    initial begin
        $display("N Z V | GE LE GT LT");
        $display("-----");

        for (int i=0; i<8; i++) begin
            {N,Z,V} = i;
            #1;
            $display("%b %b %b | %b %b %b %b", N, Z, V, GE, LE, GT,
LT);
        end

        $finish;
    end
endmodule

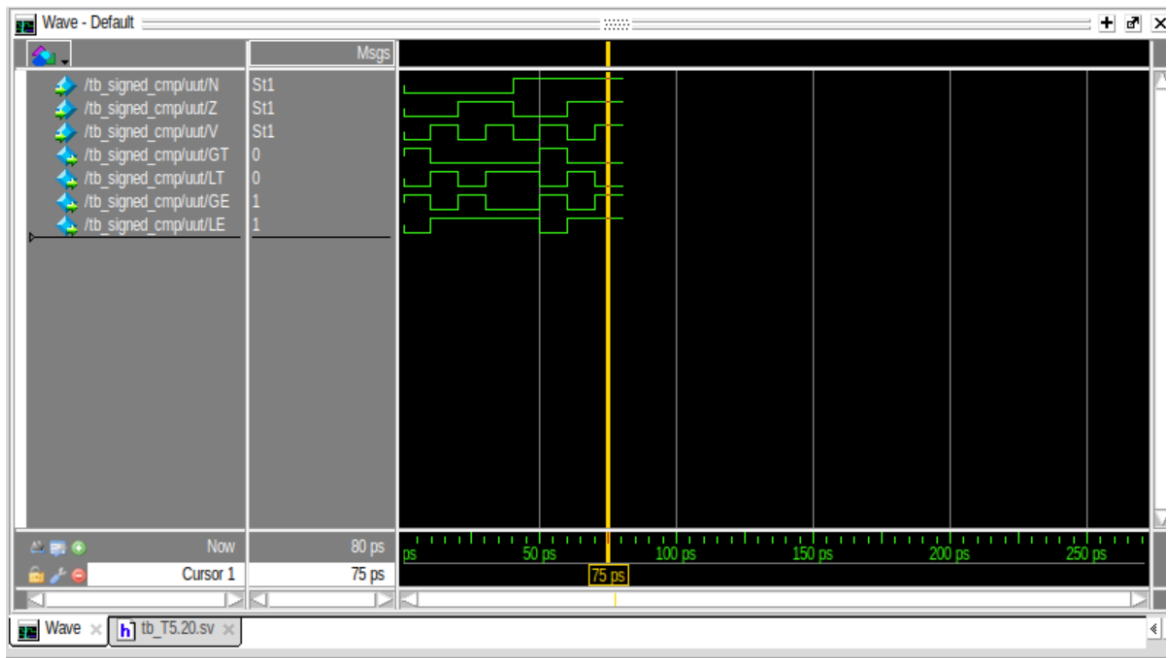
```



```

# Loading work.tb_signed_cmp
# Loading work.signed_cmp
add wave -position insertpoint sim:/tb_signed_cmp/uut/*
VSIModel -run -all
# N Z V | GE LE GT LT
# -----
# 0 0 0 | 1 0 1 0
# 0 0 1 | 0 1 0 1
# 0 1 0 | 1 1 0 0
# 0 1 1 | 0 1 0 1
# 1 0 0 | 0 1 0 1
# 1 0 1 | 1 0 1 0
# 1 1 0 | 0 1 0 1
# 1 1 1 | 1 1 0 0
# ** Note: $finish : /home/zrheng/Documents/DOCA2025Fall/Homework_Chap5/tb_T5.20.vv(24)
# Time: 80 ps Iteration: 0 Instance: /tb_signed_cmp
# 1
# Break in Module tb_signed_cmp at /home/zrheng/Documents/DOCA2025Fall/Homework_Chap5/tb_T5.20.vv line 24
VSIModel>

```



T5.32

(a) 无符号数，最小值是0（全0）。最大值是 $2^{11} + 2^{10} + \dots + 2^0 + 2^{-1} + \dots + 2^{-12}$

(b) 有符号数，最大值是 $2^{10} + 2^9 + \dots + 2^0 + 2^{-1} + \dots + 2^{-12}$ ，最小值是最大值的相反数

(c) 有符号数，最大值和(b)一样，最小值是 $-2^{11} = -2048$

T5.33

(a) 0x8D90

(b) 0x2A50

(c) 0x8914

T5.47

思路：在正式开始做之前，不妨先回忆一下IEEE754的浮点数表示方法：

$$num = (-1)^S \times 1.M \times 2^{E-127}$$

这里，我们只考虑最简单的情况，即浮点数都是规格化的、都是正数、且向0舍入。

浮点数加法主要分为三个阶段：对齐阶码、尾数相加、舍入输出结果

不妨设：

$$A = 1.M_a \times 2^{E_a-127}, B = 1.M_b \times 2^{E_b-127}$$

首先进行阶码的对齐：右移尾数较小的那个数 $|E_a - E_b|$ 位，使二者阶码相等。右移后可能丢失一些低位。待阶码对齐之后，进行尾数的直接相加。相加结果若 ≥ 2 ，则右移一位，尾数+1；若 < 1 ，则左移一位，尾数-1。舍入的话，直接丢弃低位即可。最后再表示成IEEE754标准即可。代码如下：

```

module f_add(
    input logic [31:0] a,b,
    output logic [31:0] res
);

logic [7:0] exp_a,exp_b;
logic [23:0] m_a,m_b;

assign exp_a = a[30:23];
assign exp_b = b[30:23];
assign m_a = {1'b1,a[22:0]};
assign m_b = {1'b1,b[22:0]};

logic [7:0] exp_diff;
logic [23:0] m_a_aligned,m_b_shifted;
assign exp_diff = (exp_a > exp_b) ? (exp_a - exp_b) : (exp_b - exp_a);
assign m_b_shifted = (exp_a > exp_b) ? (m_b >> exp_diff) : m_b;
assign m_a_aligned = (exp_a > exp_b) ? m_a : (m_a >> exp_diff);

logic [24:0] m_sum;
assign m_sum = m_a_aligned + m_b_shifted;

logic [23:0] m_norm;
logic [7:0] exp_res;
always_comb begin
    if (m_sum[24]) begin
        m_norm = m_sum[24:1];
        exp_res = ((exp_a > exp_b)?exp_a:exp_b) + 1;
    end
    else begin
        m_norm = m_sum[23:0];
        exp_res = (exp_a > exp_b)?exp_a:exp_b;
    end
end

assign res = {1'b0,exp_res,m_norm[22:0]};

endmodule

```

接下来给出测试

```

`timescale 10ps/1ps

module tb_f_add;

    logic [31:0] a, b, res;

    f_add dut(.a(a), .b(b), .res(res));

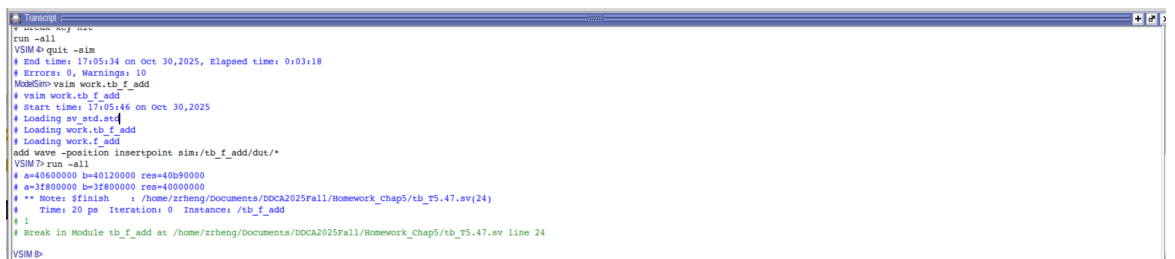
    initial begin
        // 例如 3.5 + 2.25
        a = 32'h40600000; // 3.5
        b = 32'h40120000; // 2.25
        #1;
        $display("a=%h b=%h res=%h", a, b, res);
        // 期望结果 5.75 -> 0x40b80000

        // 测试 1.0 + 1.0
        a = 32'h3f800000; // 1.0
        b = 32'h3f800000; // 1.0
        #1;
        $display("a=%h b=%h res=%h", a, b, res);
        // 期望结果 2.0 -> 0x40000000

        $finish;
    end

endmodule

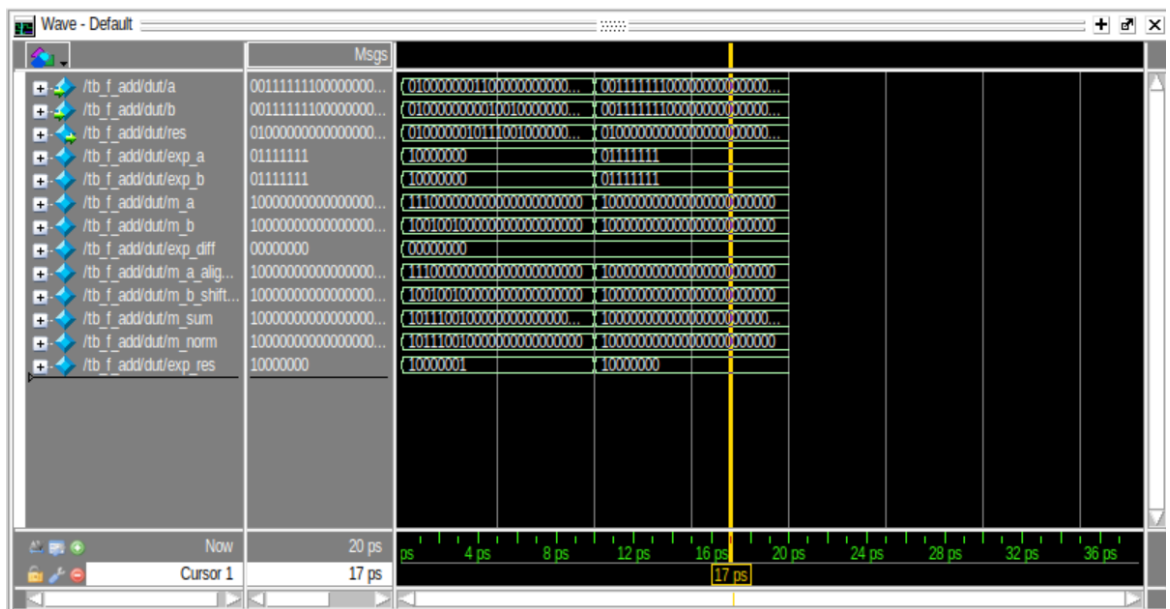
```



```

Transcript
run -all
VSIW > quit -sim
# End time: 17:05:34 on Oct 30, 2025, Elapsed time: 0:03:18
# Errors: 0, Warnings: 10
# ModeSmp vsim work.tb_f_add
# vsim work.tb_f_add
# Start time: 17:05:46 on Oct 30, 2025
# Loading sv_std.std
# Loading work.tb_f_add
# Loading work.f_add
add wave -position insertpoint sim:/tb_f_add/dut/*
VSIW > run -all
# a=40600000 b=40120000 res=40b80000
# a=3f800000 b=3f800000 res=40000000
** Note: $finish : /home/zrzheng/Documents/DDCA2025Fall/somework_chap5/tb_75.47.sv(24)
# Time: 20 ps Iteration: 0 Instance: /tb_f_add
# 1
# Break in Module tb_f_add at /home/zrzheng/Documents/DDCA2025Fall/somework_chap5/tb_75.47.sv line 24
VSIW >

```



测试结果似乎出了一些问题。计算第一个样例 $3.5+2.25$ 时，理应输出 $0x40b80000$ ，但实际输出是 $0x40b90000$ 。

T5.48

思路：有了上一道题的基础，我们继续实现一个复杂一点的浮点数乘法器。此时默认输入都是正数、规格化的浮点数、同时向0舍入。首先从32位的输入中提取出 exp （23-30位）， m （0-22位）。同时构造24位的 $M_a = 1|m_a$ ， $M_b = 1|m_b$ 。这一步是为了补全IEEE754表示法中默认的那个1。接下来开一个48bits的中间变量 P 来进行 M_a, M_b 的相乘（无符号乘法），再将二者的阶码进行相加得到 $E_{sum} = E_a + E_b - 127$ (bias)。这时回过头看乘出来的 P ，考虑其可能的范围其实是在1-4之间。若 $P[47] = 1$ （此时意味着 P 在2-4之间），则需要将 $E+1$ ，同时 P 右移1位。否则是合理的范围，不用移位。最后输出的是 $0(\text{sign})|E_{sum}|P[46:24]$ 。代码如下：

```

module fmul(
    input logic [31:0] a,
    input logic [31:0] b,
    output logic [31:0] y
);

    logic [7:0] ea, eb;
    logic [22:0] fa, fb;
    logic sa, sb;
    assign sa = a[31];
    assign sb = b[31];
    assign ea = a[30:23];
    assign eb = b[30:23];
    assign fa = a[22:0];
    assign fb = b[22:0];

    logic [23:0] ma, mb;
    assign ma = (ea == 8'd0) ? 24'd0 : {1'b1, fa};
    assign mb = (eb == 8'd0) ? 24'd0 : {1'b1, fb};
    //calc new m (48 bits)
    logic [47:0] p;
    assign p = ma * mb;

    //calc new e (max 9 bits)
    localparam int BIAS = 127;
    logic signed [9:0] e_sum_signed;
    assign e_sum_signed = $signed({2'b00,ea}) + $signed({2'b00,eb}) -
BIAS;

    //trans to IEEE
    logic p47;
    assign p47 = p[47];

    //trans e
    logic signed [9:0] e_norm;
    assign e_norm = p47 ? (e_sum_signed + 1) : e_sum_signed;

    //overflow
    logic [7:0] exp_out;
    logic [22:0] frac_out;

    logic [47:0] p_shifted;
    assign p_shifted = p47 ? (p >> 1) : p;

    logic [22:0] frac_field;
    assign frac_field = p_shifted[45:23]; // truncation -> towards zero

```



```

always_comb begin
    exp_out = 8'd0;
    frac_out = 23'd0;

    if ((ma == 24'd0) || (mb == 24'd0)) begin
        exp_out = 8'd0;
        frac_out = 23'd0;
    end
    else begin
        if (e_norm <= 0) begin
            exp_out = 8'd0;
            frac_out = 23'd0;
        end
        else if (e_norm >= 255) begin
            exp_out = 8'd255;
            frac_out = 23'd0;
        end
        else begin
            exp_out = e_norm[7:0];
            frac_out = frac_field;
        end
    end
end

logic s_out;
assign s_out = sa ^ sb;

assign y = { s_out, exp_out, frac_out };

endmodule

```

下面给出对应的测试：

```

`timescale 10ps / 1ps

module tb_fmulo();

    logic [31:0] a, b;
    logic [31:0] result;
    logic clk;

    fmulo dut (
        .a(a),
        .b(b),
        .y(result)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin

        a = 32'h3F800000; // 1.0
        b = 32'h40000000; // 2.0
        #10;
        $display("1.0 * 2.0 = %h (expected 0x40000000)", result);

        a = 32'h40400000; // 3.0
        b = 32'h40800000; // 4.0
        #10;
        $display("3.0 * 4.0 = %h (expected 0x41400000)", result);

        #10;
        $finish;
    end

endmodule

```

```
Library: Project: tb_fm1
Wave: tb_fm1_48

# Error loading design
# End time: 19:16:35 on Oct 30, 2025, Elapsed time: 0:00:00
# Errors: 1, Warnings: 8
# Compile of tb_fm1_48.v was successful.
# ModeSim vaim work.tb_fm1
# vaim work.tb_fm1
# Start time: 19:17:12 on Oct 30, 2025
# Loading sv_std.std
# Loading work.tb_fm1
# Loading work.tb_fm1
add wave -position insertpoint sim:/tb_fm1/dut/*
VSI: run -all
# 1.0 * 2.0 = 40000000 (expected 0x40000000)
# 3.0 * 4.0 = 41400000 (expected 0x41400000)
# ** Note: sfinish : /home/zrheng/Documents/DOCA2025Fall/Homework_chap5/tb_fm1_48.v(37)
# Time: 300 ps Iteration: 0 Instance: /tb_fm1
# 1
# Break in Module tb_fm1 at /home/zrheng/Documents/DOCA2025Fall/Homework_chap5/tb_fm1_48.v line 37
VSI: run
```

